A Comparison of Particle Swarm Optimization (PSO) and the Genetic Algorithm (GA) on the Ackley Benchmark Function

Experimental Report

Sadra Ghofran
Email: SadraGhofran544@gmail.com / Mail@SadraGhofran.ir
Website: SadraGhofran.ir
GitHub Repository: github.com/einmensch1847/PSO-vs-GA-Ackley-Benchmark

---

1. Introduction

Optimization has long been one of the fundamental challenges in engineering, mathematics, and artificial intelligence. Many real-world problems today, such as training neural networks, designing structures, and routing, are complex and nonlinear by nature, and finding the best possible solution to them with classical mathematical methods is extremely difficult, if not impossible. For this reason, researchers have turned to intelligent algorithms: methods inspired by natural phenomena that explore the search space in a guided way in order to arrive at an optimal solution.

Among these, **Particle Swarm Optimization** (**PSO**) and the **Genetic Algorithm** (**GA**) are two of the most popular and widely used optimization algorithms. PSO is inspired by the collective behavior of bird flocks and fish schools and relies on the interaction and shared memory of a population of particles to find a solution, whereas GA is built on Darwin's theory of evolution. Although both algorithms are capable of solving optimization problems, their behavior differs markedly when they face complex problems, and high-dimensional problems in particular.

A rigorous and scientific assessment of these differences requires a standard experimental testbed. The **Ackley function** is one of the most challenging benchmark functions in the field of optimization. It features a single large central basin (the global minimum) surrounded by thousands of small, deceptive basins, which makes it a demanding test of an algorithm's ability to escape local minima.

The purpose of this report is to implement and comprehensively compare PSO and GA on the Ackley function. The comparison is carried out across a range of dimensions (from 1 to 100) and is based on three key criteria: **success rate**, which measures the percentage of runs that reach an acceptable solution; **root mean square error** (**RMSE**), which measures accuracy; and **execution time**, which measures speed. In what follows, we first describe the methodology and experimental settings, then present the results in tables and charts, and finally analyze the behavior of both algorithms as the dimensionality of the problem increases.

2. Preliminaries

Before presenting the results and their analysis, the fundamental concepts and evaluation measures used in this report must be defined precisely. This section establishes the scientific framework that governs the experiment.

2.1 Why Do We Need Optimization Algorithms?

If a problem is simple, it can be solved with classical mathematical methods. Most real-world problems, however, are complex and multidimensional. To appreciate this complexity, consider a problem with 20 dimensions in which only 1,000 points are examined along each dimension. The total number of points to be searched is then $1000^{20} = 10^{60}$, a number far greater than the number of stars in our galaxy. In a search space this vast, exhaustive blind search is impossible. We therefore need intelligent algorithms that move purposefully toward the optimal solution without examining the entire space.

2.2 The Ackley Test Function

Evaluating the performance of algorithms on real-world problems is of limited use, because in such problems the exact optimal solution is not known in advance, and it is impossible to tell whether the algorithm has actually reached the correct answer. For this reason, **test functions** are used. These are artificial mathematical problems whose optimal solutions are known beforehand and which can simulate a variety of difficulties.

The Ackley function is one of the best-known and most challenging of these functions. Its distinguishing feature is a single large central basin (the global minimum) surrounded by thousands of small, deceptive basins, so algorithms easily become trapped in local optima. The global minimum of the function is zero. The function is defined as:

$$f(x) = -20 \cdot exp\left(-0.2\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}\right) - exp\left(\frac{1}{n}\sum_{i=1}^{n} cos(2\pi x_i)\right) + 20 + e$$

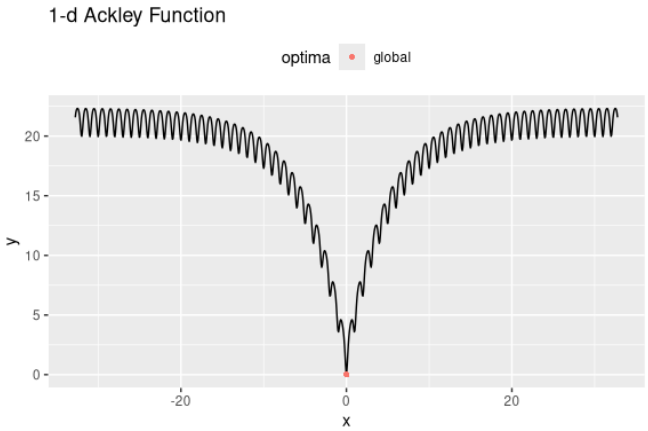


Figure 1. The one-dimensional Ackley function. Source: cran.r-project.org/web/packages/ecr/vignettes/introduction.html

2.3 Absolute Error

Absolute error is the distance between the solution found by the algorithm (Y) and the true, exact optimum of the function (Y^*):

$$Absolute\ Error\ = |Y\ -\ Y^*|$$

The smaller this value, the more accurately the algorithm has performed.

2.4 Threshold

The threshold is a predetermined number that specifies whether the obtained solution is good enough. The rule is simple:

- **Success:** if Absolute Error ≤ Threshold, the run is considered successful.
- **Failure:** if Absolute Error > Threshold, the run is considered a failure.

Why does the threshold differ across dimensions? As the dimensionality of a problem grows, the search space expands exponentially and the problem becomes harder, so algorithms naturally lose accuracy in high dimensions. If a very strict threshold such as 0.001 were imposed on every dimension (even n = 100), neither algorithm would succeed and the comparison would be meaningless. To keep the comparison feasible and to preserve a distinction between the two algorithms, the threshold is therefore relaxed somewhat as the dimensionality increases.

2.5 Success Rate

This measure expresses the ratio of successful runs to the total number of runs, as a percentage:

$$Success\ Rate\ = \left(\frac{Number\ of\ successful\ runs}{Total\ number\ of\ runs}\right) \times 100$$

It indicates the stability and reliability of an algorithm.

2.6 Root Mean Square Error (RMSE)

To assess the overall accuracy of an algorithm across multiple runs, the root mean square error is used:

$$RMSE\ =\ \sqrt{\frac{1}{N}\sum_{i=1}^{N}(Y_i\ -\ Y^*)^2}$$

where N is the number of runs and Y_i is the solution found in the i-th run. Unlike a simple mean, squaring the errors gives larger errors greater weight; RMSE is therefore a more stringent measure of how accurately the algorithm performs. The smaller this value, the higher the accuracy of the algorithm.

3. Experimental Design

3.1 Task Description

The experiment consists of running the PSO and genetic algorithms ten times each on the Ackley function across a range of dimensions (n = 1, 2, 3, 5, 10, 20, 30, 40, 50, and 100), and comparing them on three criteria: the percentage of successful runs, accuracy, and speed.

3.2 Pseudocode

The evaluation procedure is outlined below. The full implementation is available in the project's GitHub repository (github.com/einmensch1847/PSO-vs-GA-Ackley-Benchmark).

```
For each dimension dim in [1, 2, 3, 5, 10, 20, 30, 40, 50, 100]:
    threshold ← GetThreshold(dim)
    ── PSO ──
    for run = 1 to 10:
        time_start ← current time
        Y ← result of running PSO on the Ackley function (dimension = dim)
        time_end ← current time
        time_run ← time_end − time_start
        error ← |Y − 0|                        // Y* = 0
        success ← (error ≤ threshold)          // True / False
        store Y, time_run, success
    success_rate_PSO ← (number of successes / 10) × 100
    rmse_PSO ← sqrt( (1/N) · Σ (Y_i − Y*)² )      // Y* = 0
    time_PSO ← mean of the ten run times
    ── GA ──
    (identical procedure for the Genetic Algorithm)
```

3.3 Initial Threshold Values

The threshold was initially determined from the problem dimension by a piecewise rule, summarized in Table 1.

Table 1. Initial threshold values by dimension.

Dimension (n)	Threshold
$n \leq 5$	0.001
$5 < n \leq 50$	0.01
$n > 50$	0.1

4. Results

4.1 Worked Example: n = 1 (Threshold = 0.001)

Table 2 lists the ten individual runs of each algorithm in one dimension. Because the true optimum is Y* = 0, the absolute error of each run equals the value of Y.

Table 2. Individual runs of PSO and GA on the one-dimensional Ackley function.

	PSO			GA		
Run	**Y**	**Error**	**Time (ms)**	**Y**	**Error**	**Time (ms)**
1	4.44×10^{-16}	4.44×10^{-16}	18	4.44×10^{-16}	4.44×10^{-16}	46.6
2	4.44×10^{-16}	4.44×10^{-16}	5.2	4.44×10^{-16}	4.44×10^{-16}	17.6
3	4.44×10^{-16}	4.44×10^{-16}	5.1	4.44×10^{-16}	4.44×10^{-16}	18.9
4	4.44×10^{-16}	4.44×10^{-16}	5.2	4.44×10^{-16}	4.44×10^{-16}	27.7
5	4.44×10^{-16}	4.44×10^{-16}	4.9	4.44×10^{-16}	4.44×10^{-16}	29.5
6	4.44×10^{-16}	4.44×10^{-16}	5.1	4.44×10^{-16}	4.44×10^{-16}	34
7	4.44×10^{-16}	4.44×10^{-16}	8	4.69×10^{-10}	4.69×10^{-10}	21.2
8	4.44×10^{-16}	4.44×10^{-16}	6.9	4.44×10^{-16}	4.44×10^{-16}	13.5
9	4.44×10^{-16}	4.44×10^{-16}	5.4	4.44×10^{-16}	4.44×10^{-16}	19.6
10	4.44×10^{-16}	4.44×10^{-16}	4.7	4.44×10^{-16}	4.44×10^{-16}	11.2

PSO. Success rate = 100%. Mean execution time = 6.8 ms. The RMSE is:

$$RMSE = \sqrt{\frac{10 \times (4.44 \times 10^{-16})^2}{10}} = 4.44 \times 10^{-16}$$

GA. Success rate = 100%. Mean execution time = 24 ms. The RMSE is:

$$RMSE = \sqrt{\frac{9 \times (4.44 \times 10^{-16})^2 + (4.69 \times 10^{-10})^2}{10}} = 1.48 \times 10^{-10}$$

4.2 Results Across All Dimensions

The same procedure was repeated for every dimension. The results are summarized in Tables 3 to 5.

Table 3. Success rate (%) of PSO and GA by dimension.

Dimension (n)	Threshold	PSO (%)	GA (%)
1	0.001	100	100
2	0.001	100	100
3	0.001	100	100
5	0.001	100	100
10	0.01	100	0
20	0.01	100	0
30	0.01	90	0
40	0.01	70	0
50	0.01	10	0
100	0.1	0	0

Table 4. RMSE of PSO and GA by dimension.

Dimension (n)	Threshold	PSO RMSE	GA RMSE
1	0.001	4.44×10^{-16}	1.48×10^{-10}
2	0.001	4.44×10^{-16}	4.44×10^{-16}
3	0.001	4.44×10^{-16}	4.73×10^{-14}
5	0.001	3.35×10^{-15}	2.78×10^{-4}
10	0.01	5.66×10^{-15}	4.68×10^{-1}
20	0.01	1.43×10^{-10}	3.98
30	0.01	3.65×10^{-1}	6.19
40	0.01	7.55×10^{-1}	9.63
50	0.01	1.64	1.2×10^{1}
100	0.1	2.47	1.69×10^{1}

Table 5. Mean execution time (ms) of PSO and GA by dimension.

Dimension (n)	PSO (ms)	GA (ms)
1	6.8	24
2	12.05	22.72
3	13.03	28.48
5	15.44	23.62
10	41.17	52.71
20	110.32	94.06
30	179.82	180.09
40	281.25	274.36
50	394.14	384.64
100	1965.95	1907.27

5. Discussion

5.1 Success Rate

Below n = 10, both algorithms perform well. From n = 10 onward, GA collapses completely, while PSO holds up through n = 40: it remains at 100% for n = 10 and n = 20, and then declines to 90% at n = 30 and 70% at n = 40. At n = 50, PSO also nearly fails (10%), and at n = 100 neither algorithm succeeds under these thresholds.

5.2 RMSE

PSO achieves an RMSE that is lower than or equal to that of GA in every dimension. Even at n = 100, where both algorithms fail the threshold, PSO's error is 2.47 compared with approximately 16.9 for GA. PSO is therefore not only the more successful algorithm; when it does fail, it fails more gracefully, whereas GA tends either to reach the solution or to fail outright.

5.3 Execution Time

In small dimensions (n = 1 to 10), PSO is considerably faster, because each PSO iteration is simpler than a GA iteration, which involves selection, crossover, and mutation. At n = 1, PSO is roughly three times faster.

In larger dimensions ($n \geq 30$), the execution times become almost identical. In these dimensions the cost of evaluating the Ackley function exceeds the cost of the algorithm itself; that is, the bottleneck is not the algorithm but the evaluation of the objective function.

5.4 Overall Comparison

Taking all three criteria together, PSO is the stronger algorithm in this experiment: it achieves higher success rates and lower errors across the tested dimensions, at an execution time that is lower than, or comparable to, that of GA.

5.5 Effect of Increasing Dimensionality

As the dimensionality grows, PSO's success rate declines from 100% to 90% at n = 30 and lower thereafter, its error rises from practically zero (on the order of machine precision) to 2.47, and its execution time increases steadily. GA, in contrast, drops abruptly from n = 10 onward. In short, both algorithms deteriorate as the dimensionality increases, but PSO deteriorates gradually while GA deteriorates abruptly.

6. Threshold Calibration

One issue that deserves attention is that the threshold was not set appropriately. The function described in Section 3.3 (Table 1) was used for this purpose. The resulting success rates are shown in Figure 2.

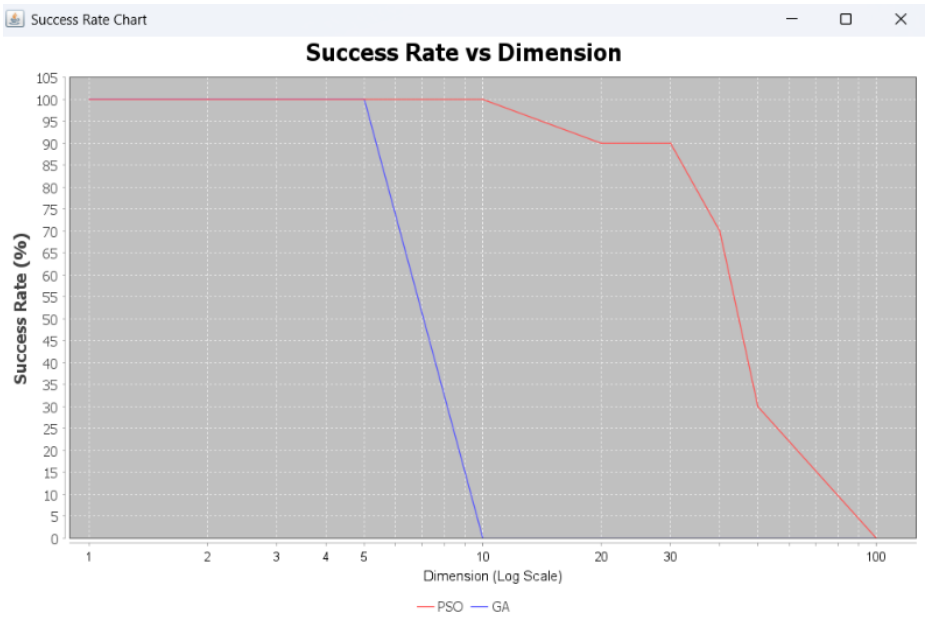


Figure 2. Success rate versus dimension under the initial thresholds (PSO in red, GA in blue).

However, because these values are determined solely by the dimension, both algorithms fail at high dimensions such as n = 100. To remedy this problem, new threshold values were

determined by trial and error so that the algorithms could be compared more meaningfully. The new values are given in Table 6.

Table 6. Revised threshold values by dimension.

Dimension (n)	Threshold
$n \leq 5$	0.001
$n = 10$	0.05
$n = 20$	0.05
$n = 30$	0.5
$n = 40$	1.0
$n = 50$	2.0
$n \geq 100$	5.0
any other n	0.05

The success results changed considerably. Under the revised thresholds, PSO remains at 100% up to n = 20, falls to about 80% at n = 30 and 60% at n = 40, and returns to 100% at n = 50 and n = 100, while GA remains at 0% from n = 10 onward (Figure 3).

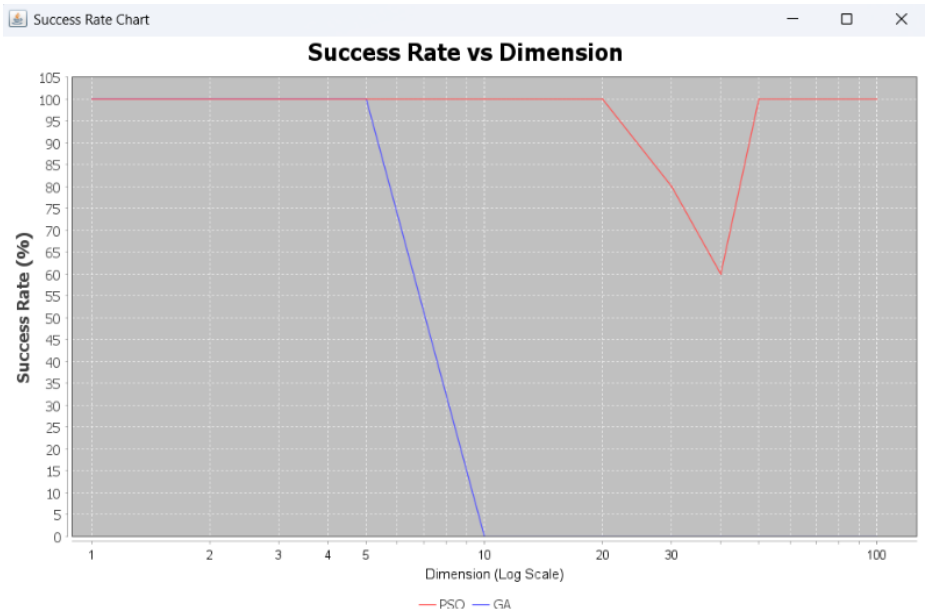


Figure 3. Success rate versus dimension under the revised thresholds (PSO in red, GA in blue).

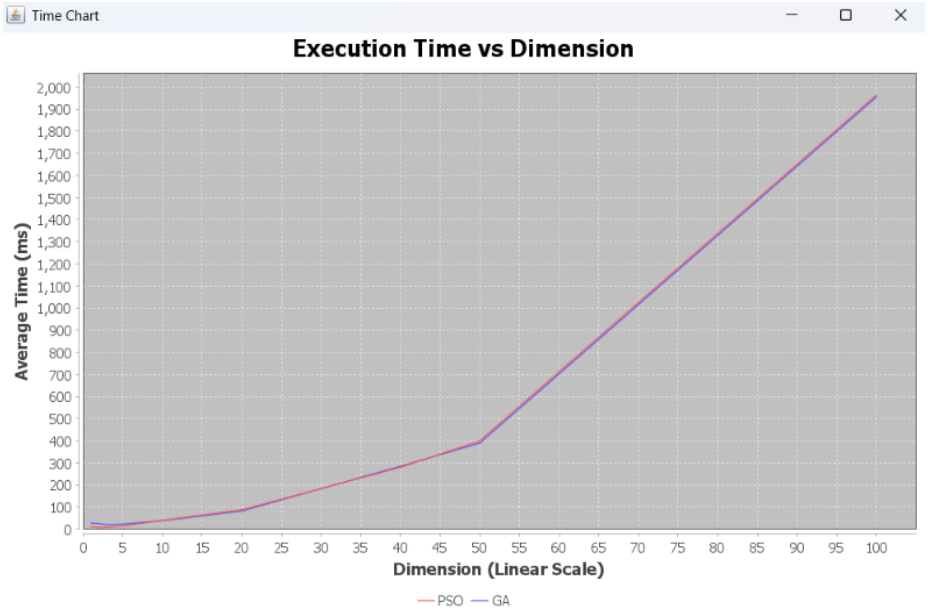


Figure 4. RMSE versus dimension (logarithmic scale on both axes; PSO in red, GA in blue).

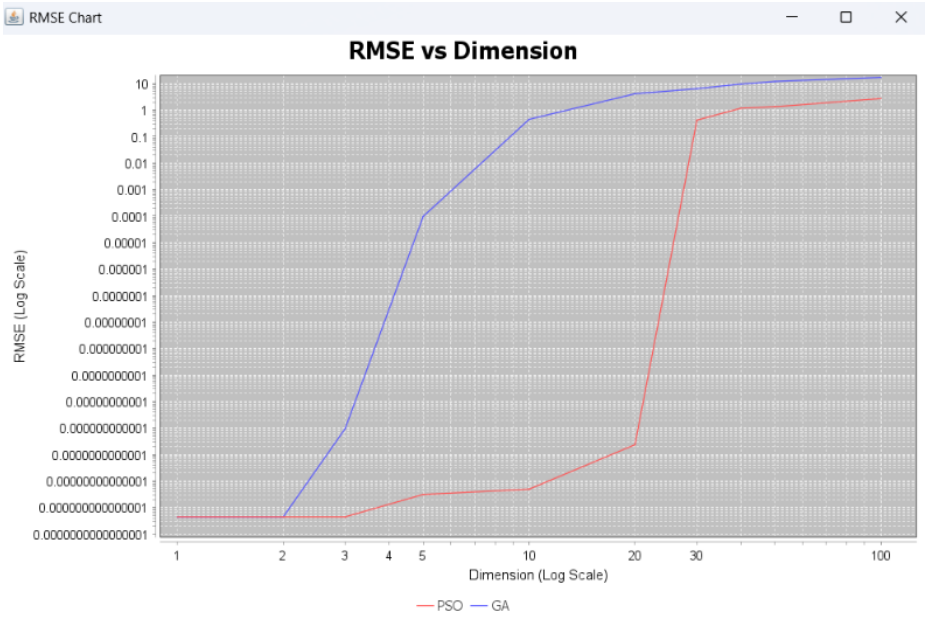


Figure 5. Mean execution time versus dimension (linear scale; PSO in red, GA in blue).

Resources

GitHub repository: github.com/einmensch1847/PSO-vs-GA-Ackley-Benchmark
Source code: sadraghofran.ir/download_file.php?id=180